

NCSAC: Effective Neural Community Search via Attribute-augmented Conductance

Longlong Lin, Quanao Li, Miao Qiao, Zeli Wang, Jin Zhao, Rong-Hua Li, Xin Luo, Tao Jia

Abstract—Identifying locally dense communities closely connected to the user-initiated query node is crucial for a wide range of applications. Existing approaches either solely depend on rule-based constraints or exclusively utilize deep learning technologies to identify target communities. Therefore, an important question is proposed: can deep learning be integrated with rule-based constraints to elevate the quality of community search? In this paper, we affirmatively address this question by introducing a novel approach called Neural Community Search via Attribute-augmented Conductance, abbreviated as NCSAC. Specifically, NCSAC first proposes a novel concept of attribute-augmented conductance, which harmoniously blends the (internal and external) structural proximity and the attribute similarity. Then, NCSAC extracts a coarse candidate community of satisfactory quality using the proposed attribute-augmented conductance. Subsequently, NCSAC frames the community search as a graph optimization task, refining the candidate community through sophisticated reinforcement learning techniques, thereby producing high-quality results. Extensive experiments on six real-world graphs and ten competitors demonstrate the superiority of our solutions in terms of accuracy, efficiency, and scalability. Notably, the proposed solution outperforms state-of-the-art methods, achieving an impressive F1-score improvement ranging from 5.3% to 42.4%. For reproducibility purposes, the source code is available at <https://github.com/longlonglin/ncsac>.

Index Terms—Community search, cohesive subgraphs, and graph neural networks

I. INTRODUCTION

A community is defined as a subgraph characterized by internal cohesiveness and external sparsity. Community Search (CS) aims to identify the specific community that exhibits a strong correlation with the user-initiated query nodes [1]. The importance of CS lies in its wide range of practical applications, including social e-commerce and uncovering personal backgrounds [1].

Existing CS models can be generally classified into two categories (details in Section VII): rule-based CS methods

[2]–[13] and learning-based CS methods [14]–[21]. The former typically identifies the resultant community by utilizing predefined structural constraints (e.g., subgraph-based k -core/truss [2], [3] or optimization-based density/modularity [5]–[8], [22]). However, they impose cumbersome constraints on the target community, leading to structural inflexibility. For instance, some nodes within the ground-truth community (such as boundary nodes) may struggle to meet these constraints, causing users to question the model’s accuracy [14]. Fortunately, learning-based CS methods can relieve the inflexibility of rule-based CS methods. In particular, one popular methodology is to define the CS problem as a node classification task [14]–[18]. These methods employ a two-stage framework. Initially, the model is trained using semi-supervised or unsupervised techniques to generate node embedding vectors. Subsequently, these embedding vectors are utilized to evaluate the probability of each vertex belonging to the target community. Another methodology is to frame the CS problem as a generative task, exemplified by CommunityAF [19]. Specifically, it first utilizes the Graph Neural Networks (GNNs) [23] to derive node representations and subsequently employs autoregressive flow to generate the community from the query node. In contrast to node classification models, generative models offer greater flexibility and eliminate concerns regarding community connectivity [19].

However, existing methods rely either entirely on rule-based constraints or solely leverage deep learning techniques for identifying target communities. Thus, a significant inquiry arises: how can deep learning be combined with traditional rule-based constraints to enhance the accuracy of CS? In this paper, we answer this question affirmatively. Two significant challenges are involved:

Challenge 1: How to identify a coarse candidate community of satisfactory quality? The most straightforward method is to use the traditional rule-based CS models. However, this technique exhibits significant limitations. First, they only consider the internal cohesiveness and ignore the external sparsity, which is an important factor in measuring the quality of a community [24]–[26], resulting in unsatisfactory quality. Second, they only optimize the topology and ignore important node attributes, further limiting their performance. Thus, our first challenge is to identify a candidate community that accounts for both the (internal and external) structural proximity and attribute similarity.

Challenge 2: How to refine the candidate community? After identifying the candidate community, it still lacks flexibility and robustness, as it remains a predefined subgraph. Recently, *ALICE* [27] and *TransZero* [18] have used GNNs to refine

Longlong Lin, Quanao Li, and Xin Luo are with the College of Computer and Information Science, Southwest University. Email: {longlonglin; luoxin}@swu.edu.cn, quanaoli174@gmail.com.

Miao Qiao is with the University of Auckland, New Zealand. Email: miao.qiao@aucklanduni.ac.nz.

Zeli Wang is with Chongqing University of Posts and Telecommunications. Email: zlwang@cqupt.edu.cn.

Jin Zhao is with Huazhong University of Science and Technology. Email: zjin@hust.edu.cn.

Rong-Hua Li is with Beijing Institute of Technology. Email: lironghuabit@126.com.

Tao Jia is with the College of Computer and Information Science, Chongqing Normal University and Southwest University. Email: tjia@swu.edu.cn.

Manuscript received XXX, XXX; revised XXX, XXXX.

the candidate community. However, they treat the candidate community as a new graph G_{new} and apply GNNs on G_{new} for node classification tasks. As a result, their methods are heavily dependent on the performance of G_{new} , since nodes outside G_{new} have no chance to contribute to the target community, resulting in poor quality (Section VI). Therefore, refining these outside and inside nodes of G_{new} to enhance the candidate community becomes our second challenge.

Our Contributions. To address these challenges, we propose a novel neural community search model via attribute-augmented conductance (named *NCSAC*) which consists of two principal components: the candidate community extraction module and the neural community refinement module. The former leverages the newly-proposed *attributed-augmented conductance* to identify a candidate community of satisfactory quality. Following this, the community refinement module non-trivially utilizes powerful reinforcement learning [28]–[30] to iteratively add and remove nodes starting from the candidate community, thereby refining the quality of the candidate community. More concretely, to address Challenge 1, we resort to the well-known conductance [9], [10], [26], [31] to capture the internal cohesion and external sparsity. However, the traditional conductance only models the structural information while neglecting crucial attribute information. To overcome this dilemma, we propose a novel attributed-augmented conductance metric, which seamlessly integrates the topology structure and node attributes. Therefore, within the candidate community extraction module, we initiate the process from the given query node to determine a candidate community with the (most favorable) approximate attribute-augmented conductance.

For Challenge 2, we design a neural community refinement module that frames the refinement process as a *combinatorial graph optimization* problem (Definition 8). The core idea is to improve the community quality by removing irrelevant nodes or appending new ones from the community boundaries, optimizing our objective through careful node selection. Unfortunately, the community refinement problem is NP-hard (Theorem 3), rendering direct solutions impractical. To address this, we present an efficient community refiner based on reinforcement learning. Specifically, we cast this problem as a reinforcement learning task and employ the Proximal Policy Optimization (PPO) [28], [29] technique for exploration. Besides, we design two distinct policy networks: one for predicting node additions and another for predicting node deletions. Through reinforcement learning, our approach can effectively refine the candidate community. On top of that, we introduce a termination strategy to ensure that the final community structure remains flexible and adaptable.

We conduct extensive experiments on six real-life graphs and ten competitors to evaluate the effectiveness and scalability of the proposed solutions. The empirical results show that our proposed solution can achieve an F1-score improvement ranging from 5.3% to 42.4% with comparable runtime in comparison to baselines.

TABLE I
THE FREQUENTLY USED SYMBOLS.

Symbol	Definitions
$G = (V, E, \mathcal{F})$	an undirected graph with the node attribute matrix $\mathcal{F}$
q, C_q	the query node, the community containing q
$\phi_t(\cdot), \phi_a(\cdot)$	topology-based conductance, attribute-based conductance
$\Phi(\cdot)$	attribute-augmented conductance
$L_C, L_T, \mathcal{L}(\phi)$	the contrastive loss, triple loss, PPO loss
$S, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma$	State, Action, State Transition, Reward, discount factor

TABLE II
A COMPARISON OF STATE-OF-THE-ART COMMUNITY SEARCH METHODS.
"SL" REPRESENTS SUPERVISED LEARNING, "UL" INDICATES UNSUPERVISED LEARNING, "RL" INDICATES REINFORCEMENT LEARNING, "NC" REPRESENTS NODE CLASSIFICATION, "GR" REPRESENTS GENERATION, AND "GO" INDICATES GRAPH OPTIMIZATION.

Methodologies	Internal Cohesiveness & External Sparsity	Learning Paradigm	Problem Definition
Subgraph-based [2]–[4]	×	×	N/A
Density [6]–[8]	×	×	N/A
Modularity [5]	×	×	N/A
Conductance [9]–[11]	✓	×	N/A
<i>ICS-GNN</i> [14]	×	SL	NC
<i>QD-GNN</i> [15]	×	SL	NC
<i>COCLEP</i> [16]	×	Semi-SL	NC
<i>CGNP</i> [17]	×	Semi-SL	NC
<i>TransZero</i> [18]	×	UL	NC
<i>ALICE</i> [27]	×	SL	NC
<i>CommunityAF</i> [19]	×	SL	GR
<i>NCSAC</i> (This paper)	✓	RL	GO

II. PRELIMINARIES

A. Problem Definition

Consider an undirected attributed graph $G = (V, E, \mathcal{F})$, where V is a node set of cardinality n , E is an edge set of cardinality m , and $\mathcal{F} \in R^{n \times k}$ is an attribute (feature) matrix. In particular, $\mathcal{F}_{uf} = 1$ indicates that node u possesses the attribute f , while $\mathcal{F}_{uf} = 0$ signifies that node u does not contain the attribute f . We refer to $\mathcal{F}[u] \in R^{1 \times k}$ as the attribute vector of node u . Let $N(u) = \{v | (u, v) \in E\}$ be the neighbors of node u and $d(u) = |N(u)|$ be the degree of u . We use $\mathbf{A} \in R^{n \times n}$ to denote the adjacency matrix of G , where $\mathbf{A}_{uv} = 1$ if $(u, v) \in E$, and $\mathbf{A}_{uv} = 0$ otherwise. Besides, we use $\mathbf{D} \in R^{n \times n}$ to represent the degree diagonal matrix of G , in which $\mathbf{D}_{uu} = d(u)$. Table I provides the frequently used symbols.

Definition 1 (Community Search [2]–[11]): Given an undirected attribute graph $G = (V, E, \mathcal{F})$, a score function $g(\cdot)$ (e.g., minimum degree or conductance), and a query node $q \in V$, the goal of community search is to identify a vertex set $C_q \subseteq V$ such that (1) $q \in C_q$; (2) C_q is connected; (3) $g(C_q)$ is optimal.

In this paper, we conceptualize community search as a *graph optimization problem* (Definition 8) rather than as a binary classification task, as seen in existing approaches like *ICS-GNN* [14] and *TransZero* [18]. At a high level, we first identify a coarse yet high-quality candidate community through a powerful subgraph extraction process, laying the groundwork for subsequent refinement. Building upon the extracted candidate community, we systematically refine it

by incorporating additional relevant nodes while removing extraneous ones. This iterative refinement process is designed to optimize community quality, ensuring a more accurate representation of the underlying relationships within the graph. Note that attribute community search (i.e., both attributes and nodes serve as query requests, e.g., *AQD-GNN* [15]) and multiple query nodes are orthogonal to ours and fall outside the scope of this paper, presenting potential directions for future research. Table II summarizes the state-of-the-art community search methods.

B. Graph Neural Network

Graph Neural Network (GNN) is a specialized neural network structure tailored for processing graph data [23], [32], [33]. It's engineered to encode network nodes as low-dimensional vectors, preserving both the network topology and node feature information, thus facilitating downstream tasks such as node classification and graph clustering [23], [34]. GNN governs the information propagation and update process by defining node aggregation and update functions, enabling nodes to assimilate information from their neighbors and refine their representations. This iterative process continues until the model converges: $h_u^{(l+1)} = \text{UPDATE}(h_u^{(l)}, \text{AGGREGATE}(h_v^{(l)} | v \in N(u)))$, in which $h_u^{(l)}$ indicates the representation of vertex u in layer l . $\text{AGGREGATE}(\cdot)$ is defined as a node aggregation function that weighs the representation of vertex v w.r.t. vertex u . $\text{UPDATE}(\cdot)$ is an update function that iteratively refines the features of all nodes based on the aggregated information.

C. Reinforcement Learning

Reinforcement learning (RL for short) is used to address the challenge of enabling agents to learn strategies that maximize returns or achieve specific goals through interactions with their environment [28]–[30], [35]–[37]. Typically, the interaction between an agent and its environment is modeled using a Markov Decision Process (MDP), which can be represented by a quintuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Here, $\mathcal{S}$ denotes the state space, while $\mathcal{A}$ represents the action space, encompassing the set of viable actions within a given environment. $\mathcal{P}$ signifies the state transition function, with $\mathcal{P}(s'|s, a)$ denoting the probability of transitioning from state s to state s' upon taking action a . $\mathcal{R}$ represents the reward function, typically a deterministic function crafted by humans. $\gamma \in [0, 1]$ stands for the discount factor, which is a constant. The goal of RL is to learn a policy by maximizing the expected *discounted return* $U_t = E[\sum_{i=t}^{\infty} \gamma^{i-t} \mathcal{R}_i]$, in which $\mathcal{R}_i$ is the reward for transitioning from state s_{i-1} to s_i .

III. THE PROPOSED FRAMEWORK

In this section, we propose an effective community search method, *NCSAC*, as illustrated in Figure 1. Specifically, *NCSAC* first utilizes our proposed *attribute-augmented conductance* to adaptively extract a coarse candidate community by identifying the h -hop subgraph with optimal attribute-augmented conductance. We highlight that the attribute-augmented conductance simultaneously captures both the

topological structure (i.e., internal cohesiveness and external sparsity) and the attributes of nodes. Consequently, the resulting coarse candidate community exhibits comparable community quality (Section IV). Subsequently, *NCSAC* pre-trains a graph neural network to serve as the state encoder, integrating community awareness into the state representation through our proposed contrastive loss and triplet loss. Once the coarse candidate community is encoded into the initial state by the state encoder, reinforcement learning is employed to initiate the community refinement process. At each step of refinement, the agent processes the community's state encoding and refines the community by incorporating promising nodes while simultaneously excluding noisy or irrelevant ones. The refined community is then re-encoded by the state encoder to generate the next state, enabling the agent to proceed with the subsequent refinement step. This process continues until the predefined termination strategy is satisfied (Section V).

IV. CANDIDATE COMMUNITY EXTRACTION

This section first proposes a novel concept of attribute-augmented conductance to measure the quality of a community, which seamlessly blends structural and attribute information. Then, we devise an efficient dynamic update algorithm to adaptively identify high-quality candidate communities.

A. Attribute-augmented Conductance

As previously mentioned, a crucial aspect of community search is the development of an indicator to measure community quality. To this end, we adopt the well-known free-parameter community metric, *conductance*, due to its advantageous structural properties and solid theoretical foundation [9]–[11], [25], [26], [31], [38], [39]. Unlike metrics such as k -core [40], k -truss [41], k -clique [42], and modularity [43], which measure only the internal cohesion of a community, conductance takes into account both internal cohesion and external sparsity. This dual consideration makes conductance a more robust measure of community quality [24], [26], [44].

Definition 2 (Conductance): Given a graph G and a vertex set C , the conductance of C is defined as follows.

$$\phi(C) = \frac{|\text{cut}(C, \bar{C})|}{\min\{\text{vol}(C), \text{vol}(\bar{C})\}}. \quad (1)$$

where $\bar{C}$ denotes the complement of the community, defined as $\bar{C} = V \setminus C$. The cut between community C and its complement is represented as $\text{cut}(C, \bar{C}) = \{(u, v) \mid u \in C, v \in \bar{C}\}$, while $\text{vol}(C) = \sum_{u \in C} d(u)$ indicates the total sum of node degrees within community C . Therefore, a smaller value of $\phi(C)$ suggests that the number of edges exiting community C is relatively low compared to the number of edges contained within it. Consequently, a lower value of $\phi(C)$ indicates a higher quality of community C [9], [10], [26], [31]. A more interesting and useful probabilistic explanation of the conductance is stated as follows.

Theorem 1: Given a graph G and a community C . Consider a random walk $(w_t)_{t \in \mathbb{N}}$ generated by the transition matrix

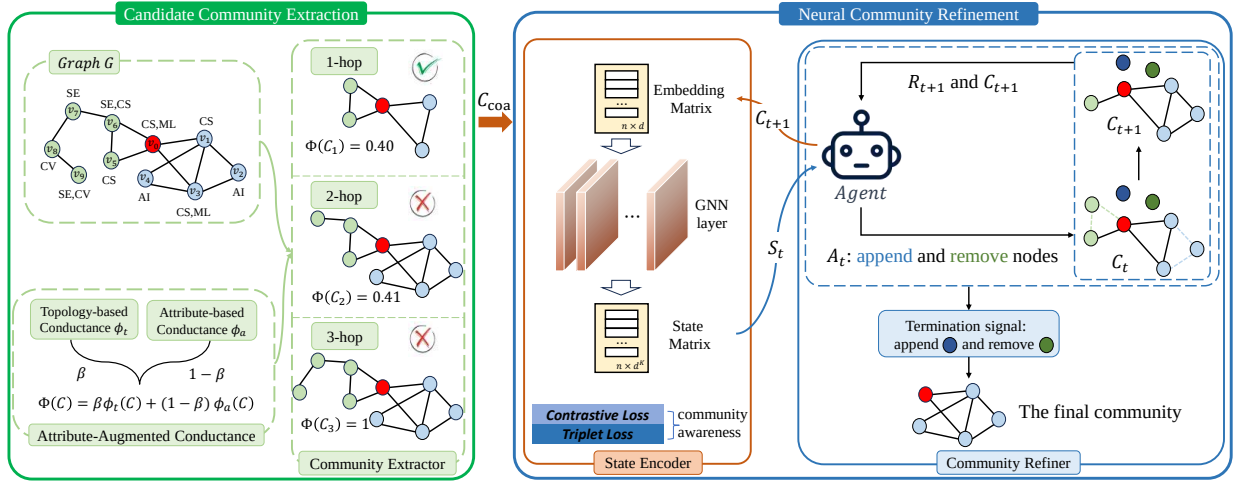


Fig. 1. The framework of our proposed NCSAC. In the candidate community extraction stage, NCSAC systematically extracts a coarse candidate community C_{coa} by leveraging our novel attribute-augmented conductance (the red node is the query node). Subsequently, we pre-train a state encoder to effectively integrate community awareness information into the state encoding. Finally, we apply non-trivial reinforcement learning techniques to flexibly refine the coarse candidate community, ultimately yielding high-quality results (best view in color).

$P(=D^{-1}A)$, and the initial state w_0 is in C (or $\bar{C}$) and is randomly chosen following the degree distribution. We have

$$\phi(C) = \max\{Pr(w_1 \in \bar{C}|w_0 \in C), Pr(w_1 \in C|w_0 \in \bar{C})\}. \quad (2)$$

Proof 1: Firstly, according to Eq. (1), we can deduce that $\phi(C) = \frac{|cut(C, \bar{C})|}{\min\{vol(C), vol(\bar{C})\}} = \max\{\frac{|cut(C, \bar{C})|}{vol(C)}, \frac{|cut(\bar{C}, C)|}{vol(\bar{C})}\}$. Then, we have $Pr(w_1 \in \bar{C}|w_0 \in C) = \sum_{u \in C} \sum_{v \in \bar{C}} \frac{d(u)}{vol(C)}$. $P_{uv} = \sum_{u \in C} \sum_{v \in \bar{C} \cap N(u)} \frac{d(u)}{vol(C)} \cdot \frac{1}{d(u)} = \sum_{u \in C} \sum_{v \in \bar{C} \cap N(u)} \frac{1}{vol(C)}$. Here, $\frac{d(u)}{vol(C)}$ denotes the probability of randomly selecting node u from community C following the degree distribution, while P_{uv} represents the probability that node u transitions to node v by one-hop random walk. Analogously, we have $Pr(w_1 \in C|w_0 \in \bar{C}) = \frac{|cut(C, \bar{C})|}{vol(\bar{C})}$. Consequently, the theorem is proved. ■

Theorem 1 offers a random walk interpretation of conductance. Specifically, the conductance of the community C is the probability that a random walker, beginning within C (or $\bar{C}$), will escape to the outside by one-hop random walk. To distinguish it from the attribute-based conductance discussed later, we redefine conductance as topology-based conductance, denoted by $\phi_t(C)$. However, $\phi_t(C)$ falls short in this context, as it fails to consider the attributes associated with the nodes. Inspired by the random walk interpretation of conductance, we propose the attribute-based conductance to capture structural proximity and attribute similarity.

Definition 3 (Attribute Edge): Given an attributed graph $G = (V, E, \mathcal{F})$, the connection between nodes u and v through a specific attribute f is considered an *attribute edge*, denoted as (u, v, f) . Namely, (u, v, f) indicates nodes u and v share attribute f (i.e., $\mathcal{F}_{uf}=1$ and $\mathcal{F}_{vf}=1$).

Definition 4 (Attribute Degree): Given an attributed graph $G = (V, E, \mathcal{F})$, the attribute degree of a node u , denoted as $d_a(u)$, is defined as the number of attribute edges connected

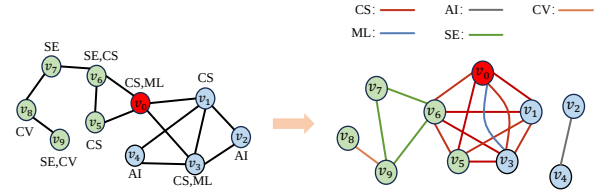


Fig. 2. From the original attribute graph to the multigraph.

to u . The formula for its calculation is presented as follows:

$$d_a(u) = \sum_{v \in V \setminus \{u\}} \mathcal{F}[u] \cdot \mathcal{F}[v]^\top. \quad (3)$$

Definitions 3 and 4 map the original attributed graph into a multigraph (i.e., two vertices can be connected by more than one edge), as depicted in Figure 2. In this multigraph, the edges correspond to attribute edges, and the degree of each node reflects its attribute degree. Unfortunately, the multigraph may be extremely dense (even close to quadratic), especially if there is an attribute value shared by a decent portion, e.g., 50%, of the nodes in the graph. Section IV-B will show how to quickly obtain attribute-augmented conductance without materializing the multigraph.

Definition 5 (Attribute-based Random Walk): Given an attribute graph G , the attribute-based transition probability from node u to node v is defined as follows:

$$P_{uv}^a = \frac{\mathcal{F}[u] \cdot \mathcal{F}[v]^\top}{d_a(u)}. \quad (4)$$

Intuitively, P_{uv}^a models the transition probability from u to v based on the attribute set of G .

Example 1: Consider the example of nodes v_0 and v_3 in Figure 2. v_0 is connected to v_3 through two attribute edges, specifically (v_0, v_3, CS) and (v_0, v_3, ML) . Besides, v_0 also has three other attribute edges, namely, (v_0, v_1, CS) , (v_0, v_5, CS) , and (v_0, v_6, CS) . Thus, the attribute degree of node v_0 equals 5 and the attribute transition probability from v_0 to v_3 is $P_{v_0 v_3}^a = \frac{2}{5} = 0.4$.

Definition 6 (Attribute-based Conductance): Given an attributed graph G and a community C , the attribute-based conductance, denoted as $\phi_a(C)$, is defined as follows:

$$\phi_a(C) = \frac{|\text{cut}_a(C, \bar{C})|}{\min\{\text{vol}_a(C), \text{vol}_a(\bar{C})\}}. \quad (5)$$

Here, $\text{cut}_a(C, \bar{C}) = \{(v_i, v_j, f) \mid v_i \in C, v_j \in \bar{C}\}$ denotes the number of attribute edges connecting community C and $\bar{C}$, while $\text{vol}_a(C) = \sum_{u \in C} d_a(u)$ represents the total sum of attribute degrees within C .

We highlight that while the definitions of attribute-based conductance $\phi_a(C)$ and topology-based conductance $\phi_t(C)$ are similar, computing $\phi_a(C)$ is significantly more complex (details in Section IV-B). Besides, analogous to Theorem 1, we can also understand attribute-based conductance from the perspective of attribute-based random walk, thereby enhancing its practical applicability. Specifically, we reformulate Eq. (5) as $\phi_a(C) = \max\{Pr(\omega_1 \in \bar{C} \mid \omega_0 \in C), Pr(\omega_1 \in C \mid \omega_0 \in \bar{C})\}$, where $(\omega_t)_{t \in \mathbb{N}}$ represents a random walk generated by the attribute-based transition matrix P^a (i.e., Eq. 4). Therefore, a smaller $\phi_a(C)$ indicates a lower probability of transitioning from community C (or $\bar{C}$) to $\bar{C}$ (or C) through the one-hop attribute-based random walk. Intuitively, this suggests that the nodes within the community largely share similar attributes, while there is limited overlap with the attributes of nodes outside the community. This observation aligns with our principles of internal attribute cohesiveness and external attribute sparsity.

Building on the concepts discussed earlier, we now introduce the definition of attribute-augmented conductance, which can capture simultaneously the topology and the attributes of the community.

Definition 7 (Attribute-augmented Conductance): Given an attributed graph G , a community C , and a preference parameter β , the attribute-augmented conductance is as follows:

$$\Phi(C) = \beta \cdot \phi_t(C) + (1 - \beta) \cdot \phi_a(C). \quad (6)$$

Remark. $\Phi(C)$ is constructed as a combination of $\phi_t(C)$ and $\phi_a(C)$ weighted by preference parameter β . This formulation captures a trade-off between topological structure and node attributes, allowing us to identify communities with distinct characteristics by varying β . In particular, when $\beta > 0.5$, the community emphasizes the interaction edges among vertices. Conversely, when $\beta < 0.5$, the focus shifts toward the attribute similarity among vertices.

B. Adaptive Community Extractor

Local search [2], [5], [42], [45], [46] is a well-established technique for identifying the community to which a query node belongs, utilizing an iterative strategy that incrementally adds nodes in a designed ordering. We emphasize that directly obtaining the optimal solution through this ordering is generally NP-hard [2], and it becomes even more complex when considering conductance [11], ultimately resulting in poor scalability. Fortunately, our principal aim is to efficiently derive a coarse candidate community with satisfactory quality, which will subsequently be refined through our proposed novel

and non-trivial reinforcement learning techniques. To this end, we let $N^h(q) = \{v \mid \text{dist}(q, v) \leq h\}$ be the h -hop neighbors of the query node q , where $\text{dist}(q, v)$ represents the shortest-path distance between nodes q and v . We designate $N^h(q)$ as the candidate community, which has been well-supported by existing literature [5], [42], [45], [46]. Intuitively, nodes that are closer to the query node are more likely to belong to the target community. This approach implicitly defines the order of node addition (i.e., by adding based on distance in batches).

Example 2: In Figure 1, we use v_0 as the query node, representing 1-hop, 2-hop, and 3-hop subgraphs as C_1 , C_2 , and C_3 , respectively. We first compute the topology-based conductance values, obtaining $\phi_t(C_1) = 0.55$, $\phi_t(C_2) = 0.33$, and $\phi_t(C_3) = 0.55$. From these values, we select C_2 as the coarse community by only using the topology-based conductance. Next, when incorporating attributes, the attribute-based conductance values are calculated as $\phi_a(C_1) = 0.25$, $\phi_a(C_2) = 0.5$, and $\phi_a(C_3) = 1$. By setting $\beta = 0.5$, indicating equal weighting of both topological structure and node attribute, the attribute-augmented conductance values are adjusted to $\Phi(C_1) = 0.40$, $\Phi(C_2) = 0.415$, and $\Phi(C_3) = 0.775$. In this case, C_1 becomes the coarse community, as it better aligns with the community definition compared to C_2 .

A failed attempt. According to Definition 7, a naive approach to calculating attribute-augmented conductance is first to construct a multigraph (Figure 2). Subsequently, it uses Eq. 5 to obtain the attribute-based conductance and then applies Eq. 6 to derive the attribute-augmented conductance. However, this naive solution has two-fold drawbacks: (1) Materializing the multigraph requires prohibitively high time and space complexities. In particular, it takes $O(kn^2)$ time and $O(kn^2)$ space to materialize the multigraph by Eq. 3, resulting in poor scalability. Where n and k are the number of nodes and attributes of G , respectively. (2) When the subgraph $N^h(q)$ extends to $N^{h+1}(q)$, calculating the attribute-augmented conductance of $N^{h+1}(q)$ from scratch entails significant redundant computations. Consequently, the naive method cannot deal with massive graphs with millions of edges (details in Section VI-C). Thus, an important challenge is quickly calculating and updating attribute-augmented conductance without materializing the multigraph.

Several in-depth observations. Analysis of Eq. 6 reveals that the main bottlenecks in calculating and updating attribute-augmented conductance lie in the efficient computation of $\text{vol}_a(C)$ and the rapid update of $|\text{cut}_a(C, \bar{C})|$. To overcome this challenge, we observe that a simple matrix-vector product can obtain the attribute degrees of all nodes. Besides, we can also quickly and dynamically update $|\text{cut}_a(C, \bar{C})|$ by simply maintaining several arrays. For convenience, we state these observations as the following lemmas.

Lemma 1: Consider an attributed graph $G = (V, E, \mathcal{F})$, for any node $u \in V$, the attribute degree of u can be reformatted as $d_a(u) = \mathcal{F}[u] \cdot (\mathbf{1}_n \mathcal{F} - \mathbf{1}_k)^\top$, in which $\mathbf{1}_n \in R^{1 \times n}$ (or $\mathbf{1}_k \in R^{1 \times k}$) is a vector with all elements equal to 1. Besides, we have $d_a = \mathcal{F} \cdot (\mathbf{1}_n \mathcal{F} - \mathbf{1}_k)^\top$.

Proof 2: By Eq. 3, we have $d_a(u) = \sum_{v \in V \setminus \{u\}} \mathcal{F}[u] \cdot \mathcal{F}[v]^\top = \mathcal{F}[u] \cdot \sum_{v \in V \setminus \{u\}} \mathcal{F}[v]^\top = \mathcal{F}[u] \cdot ((\mathbf{1}_n \mathcal{F})^\top -$

Algorithm 1 Adaptive Community Extractor

Input: An attributed graph $G = (V, E, \mathcal{F})$, a query node q , and a parameter β .

Output: The coarse candidate community C .

```

1:  $d_a \leftarrow \mathcal{F} \cdot (\mathbf{1}_n \mathcal{F} - \mathbf{1}_k)^\top$ ;  $\text{vol}_a(V) \leftarrow \sum_{v \in V} d_a(v)$ 
2:  $\text{cut} \leftarrow d(q)$ ;  $\text{vol} \leftarrow d(q)$ ;  $\text{cut}_a \leftarrow d_a(q)$ ;  $\text{vol}_a \leftarrow d_a(q)$ 
3:  $C \leftarrow \{q\}$ ;  $\text{tmp} \leftarrow \{q\}$ ;  $\hat{\Phi} \leftarrow 1$ ;  $\text{att} \leftarrow \mathcal{F}[q]$ 
4: for  $h = 1$  to  $\max\{\text{dist}(q, u) | u \in G\}$  do
5:    $\Delta(h) \leftarrow N^h(q) - N^{h-1}(q)$ 
6:   for each  $u \in \Delta(h)$  do
7:      $\text{tmp.add}(u)$ 
8:      $\text{cut} \leftarrow \text{cut} + d(u) - 2 \cdot |C \cap N(u)|$ ,  $\text{vol} \leftarrow \text{vol} + d(u)$ 
9:      $\text{cut}_a \leftarrow \text{cut}_a + d_a(u) - 2 \cdot \mathcal{F}[u] \cdot \text{att}^T$ ,  $\text{vol}_a \leftarrow \text{vol}_a + d_a(u)$ 
10:    $\text{att} \leftarrow \text{att} + \mathcal{F}[u]$ 
11:   if  $\frac{\beta \cdot \text{cut}}{\min\{\text{vol}, 2m - \text{vol}\}} + \frac{(1-\beta) \cdot \text{cut}_a}{\min\{\text{vol}_a, \text{vol}_a(V) - \text{vol}_a\}} < \hat{\Phi}$  then
12:      $\hat{\Phi} \leftarrow \frac{\beta \cdot \text{cut}}{\min\{\text{vol}, 2m - \text{vol}\}} + \frac{(1-\beta) \cdot \text{cut}_a}{\min\{\text{vol}_a, \text{vol}_a(V) - \text{vol}_a\}}$ 
13:      $C \leftarrow \text{tmp}$ 
14: return  $C$ 

```

$\mathcal{F}[u]^\top = \mathcal{F}[u] \cdot (\mathbf{1}_n \mathcal{F} - \mathcal{F}[u])^\top = \mathcal{F}[u] \cdot (\mathbf{1}_n \mathcal{F} - \mathbf{1}_k)^\top$. This is because every element in $\mathcal{F}[u]$ is either 0 or 1, $\mathcal{F}[u] \cdot \mathcal{F}[u]^\top = \mathcal{F}[u] \cdot \mathbf{1}_k^\top$. Thus, this lemma is proved. ■

Lemma 2: For a community C and any $u \in \bar{C}$, we have $\text{cut}(C \cup \{u\}, V \setminus (C \cup \{u\})) = |\text{cut}(C, \bar{C})| + d(u) - 2|\text{cut}(u, C)|$ and $\text{cut}_a(C \cup \{u\}, V \setminus (C \cup \{u\})) = |\text{cut}_a(C, \bar{C})| + d_a(u) - 2|\text{cut}_a(u, C)|$.

Proof 3: By Definition 2, we have $\text{cut}(C \cup \{u\}, V \setminus (C \cup \{u\})) = |\text{cut}(C, \bar{C})| - |\text{cut}(u, C)| + |\text{cut}(u, \bar{C})| = |\text{cut}(C, \bar{C})| + d(u) - 2|\text{cut}(u, C)|$ due to $|\text{cut}(u, C)| + |\text{cut}(u, \bar{C})| = d(u)$. Similarly, we can prove $\text{cut}_a(C \cup \{u\}, V \setminus (C \cup \{u\})) = |\text{cut}_a(C, \bar{C})| + d_a(u) - 2|\text{cut}_a(u, C)|$. Thus, this lemma is proved. ■

Fact 1: Given a community C and any $u \in \bar{C}$, based on Lemma 2, we have the following facts.

$$\phi_t(C \cup \{u\}) = \frac{|\text{cut}(C, \bar{C})| + d(u) - 2|\text{cut}(u, C)|}{\min\{\text{vol}(C) + d(u), 2m - \text{vol}(C) - d(u)\}}. \quad (7)$$

$$\phi_a(C \cup \{u\}) = \frac{|\text{cut}_a(C, \bar{C})| + d_a(u) - 2|\text{cut}_a(u, C)|}{\min\{\text{vol}_a(C) + d_a(u), \text{vol}_a(V) - \text{vol}_a(C) - d_a(u)\}}. \quad (8)$$

By Fact 1, we can know that the challenge with incremental computation of ϕ_t (or ϕ_a) is how to efficiently maintain $|\text{cut}(u, C)|$ (or $|\text{cut}_a(u, C)|$). Note that $|\text{cut}(u, C)| = |N(u) \cap C|$ is easily obtained in $O(d(u))$ time. However, calculating $|\text{cut}_a(u, C)|$ is more complicated. This is due to the fact that $|\text{cut}_a(u, C)| = |\{(u, v, f) \mid v \in C\}| = \sum_{v \in C} \mathcal{F}[u] \mathcal{F}[v]^\top$ as defined in Definition 6, which requires $O(k|C|)$ time to compute. To improve efficiency, we devise several non-trivial data structures to efficiently update ϕ_t and ϕ_a .

Implementation details. Algorithm 1 provides the pseudocode of our adaptive community extractor. Specifically, in Lines 1-3, it uses Lemma 1 to obtain the attribute degree vector d_a and initializes the candidate community C , current search space tmp , and internal attribute att . Then, by Lemma 2, it executes the incremental dynamic update process for cut , vol , cut_a , and vol_a (Lines 5-10). Lines 11-13 update the candidate community C and the optimal value $\hat{\Phi}$. Finally, Line 14 returns C as the candidate community.

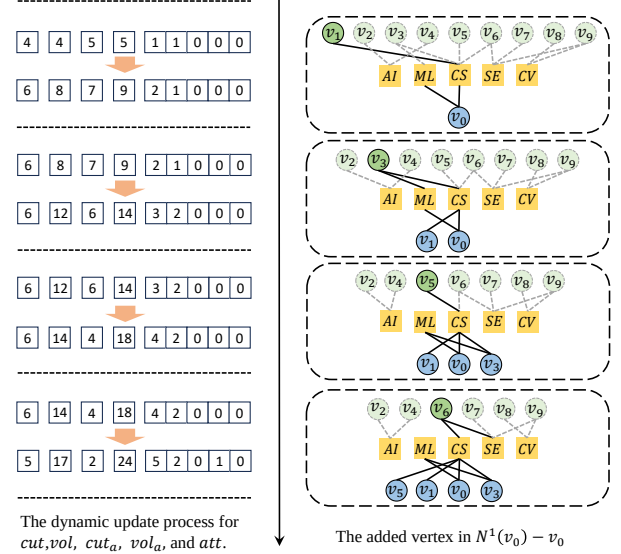


Fig. 3. Illustration of Example 3 for Algorithm 1. The left side shows the update process that occurs after each node is added. On the right side, the update of cut_a is elaborated using a node-attribute bipartite graph. The dark blue nodes indicate the current search space tmp , while the dark green nodes represent the nodes to be added (best view in color).

Example 3: Reconsider Figure 2, Figure 3 illustrates the process of extending from the query node v_0 to $N^1(v_0)$. To demonstrate the dynamic update algorithm, we focus on the addition of node v_3 (i.e., the second step). Initially, the community C consists of nodes v_0 and v_1 . The neighbors of node v_3 include v_0 , v_1 , v_2 , and v_4 , with two edges connecting v_3 to community C . Consequently, $|\text{cut}(v_3, C)| = 2$. Then cut is updated to $\text{cut} = 6 + 4 - 2 \times 2 = 6$, while $\text{cut}_a = 8 + 4 = 12$. At this juncture, the internal attribute vector att is $[2, 1, 0, 0, 0]$, signifying that two edges connecting to the CS attribute and one edge connecting to the ML attribute are associated with community C . Therefore, $|\text{cut}_a(v_3, C)| = \mathcal{F}[v_3] \cdot \text{att}^T = [1, 1, 0, 0, 0] \cdot [2, 1, 0, 0, 0]^T = 3$, representing the three attribute edges through which node v_3 is linked to community C via its own attributes (i.e., CS and ML). The updated cut_a then becomes $7 + 5 - 2 \times 3 = 6$, and $\text{vol}_a = 9 + 5 = 14$. Following the addition of node v_3 , att is updated to $[1, 1, 0, 0, 0] + [2, 1, 0, 0, 0] = [3, 2, 0, 0, 0]$. This process continues iteratively, adding nodes sequentially until all nodes (i.e. $\Delta(h)$) are added.

Theorem 2: The time complexity and space complexity of Algorithm 1 are $O(m + nk)$ and $O(m + n + k)$, respectively. k ($k \ll n$) is the attribute dimension.

Proof 4: Algorithm 1 first takes $O(nk)$ time to calculate the attribute degree vector d_a , $O(n)$ time to obtain $\text{vol}_a(V)$, cut , vol , cut_a , and vol_a (Lines 1-2). In Lines 4-13, Algorithm 1 can incrementally calculate and update attribute-augmented conductance by Fact 1. Specifically, we assume that u is the added vertex in Line 6, it takes $O(d(u))$ time to incrementally update cut and $O(k)$ time to update cut_a and att . Thus, it takes $O(\sum_{u \in V} (d(u) + k)) = O(m + nk)$ to execute Lines 4-13. As a result, Algorithm 1 takes $O(m + nk)$ time in total. For the space complexity, Algorithm 1 just needs an extra $O(n + k)$ to store d_a , C , tmp , and att . Besides, it needs $O(m + n)$ to store the input graph G . As a consequence, Algorithm 1 needs

$O(n + m + k)$ space in total. ■

V. NEURAL COMMUNITY REFINEMENT

Section IV prioritizes scalability over community quality, necessitating further refinement of the coarse candidate community. Therefore, we develop a Neural Community Refinement method, structured into two primary components. In the State Encoder component, we guide the pre-training model using two loss functions that integrate community-aware node representations as state inputs for reinforcement learning. In the Community Refiner component, we employ reinforcement learning to balance exploration and exploitation, thereby identifying nodes to be appended or removed. This process ultimately yields high-quality communities.

A. Community-aware State Encoder

In this component, our objective is to incorporate community awareness into the embedded space of nodes. To this end, we design two loss functions to achieve a strong correlation between nodes and communities: *Contrastive Loss* and *Triplet Loss*. Intuitively, an edge between two nodes suggests a high probability that they belong to the same community, and the absence of such an edge indicates otherwise. Thus, we model this relationship using contrastive loss:

$$L_C = \sum_{u,v \in V} A_{uv} \delta(h_u, h_v) + (1 - A_{uv}) \max(0, \gamma_1 - \delta(h_u, h_v)). \quad (9)$$

$\delta(\cdot)$ represents the distance metric, for which we utilize cosine distance. γ_1 is the margin hyperparameter that indicate the minimum distance. In the context of the community search task, when provided with a seed node, the remaining nodes can be classified into two distinct groups: those within the same community and those in different communities. To effectively capture this differentiation, we implement triplet loss:

$$L_T = \sum_{(q, q_+, q_-)} \max\{\delta(h_q, h_{q_+}) - \delta(h_q, h_{q_-}) + \gamma_2, 0\}. \quad (10)$$

where q , q_+ , and q_- denote the query node, the positive sample (i.e., q_+ and q are in the same community), and the negative sample (i.e., q_- and q are in different communities), respectively. γ_2 is the margin hyperparameter that indicates the minimum distance. We concurrently account for both of the aforementioned loss functions. As a consequence, the aggregate loss is formally defined as follows:

$$L = L_T + \alpha L_C. \quad (11)$$

Without loss of generality, in this paper, we utilize Graph Convolutional Network (GCN [23]) as our encoder in this component, defined as follows:

$$h_u^{l+1} = Dr(\sigma(h_u^l W_s^{l+1} + \sum_{v \in N(u)} \frac{h_v^l W^{l+1}}{\sqrt{(d(u)+1)(d(v)+1)}} + b^{l+1})). \quad (12)$$

Here, W_s^{l+1} , W^{l+1} and b^{l+1} represents the trainable weights, and $\sigma(\cdot)$ denotes a nonlinear activation function, e.g., ReLU. The parameter $Dr(\cdot)$ refers to the dropout rate, which is used to prevent overfitting. In this paper, we employ a two-layer GCN architecture.

B. RL-based Community Refiner

As previously indicated, after acquiring the candidate community, we can redefine the community search problem as a community refinement graph optimization task.

Definition 8 (Community Refinement Graph Optimization): Given a graph G and a candidate coarse community C_{coa} , along with a score function $\Upsilon(\cdot)$, the goal of community refinement is to identify a community C_{opt} with the optimal $\Upsilon(C_{opt})$ by appending or removing nodes for C_{coa} .

Theorem 3: The community refinement problem is NP-hard.

Proof 5: Let $\Pi = \{ \langle (u_1, op_1), (u_2, op_2), \dots \rangle \mid u_i \in G, op_i \text{ is append or remove} \}$ be the set of all possible refinement orderings, where (u_i, append) (resp., (u_i, remove)) means that we append (resp., remove) the node u_i for the current community. Thus, by Definition 8, we can know that the goal of community refinement is to find an ordering $\pi \in \Pi$ such that $\Upsilon(C_{coa} + \pi)$ has the optimal score value. Note that $C_{coa} + \pi$ is a new community by executing the operation π for C_{coa} . Based on the above concepts, we can reduce the well-known NP-hard problem of the maximum subset sum problem (decision version) to the community refinement problem within polynomial time. Given a set of integers $S = \{s_1, s_2, \dots, s_n\}$ and a target integer t , the subset sum decision problem is to check whether there exists a subset $S' \subseteq S$ such that the sum of the elements in S' equals t . From this, we have the following steps. (1) Graph Construction: Construct a graph G where each integer $s_i \in S$ is represented as a node, and let the candidate coarse community $C_{coa} = \emptyset$. (2) Score Function Definition: we define the score function $\Upsilon(C)$ for any community C as follows:

$$\Upsilon(C) = \begin{cases} 1, & \text{if the sum of the nodes in } C \text{ is exactly } t \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

(3) Polynomial Time Reduction: Given the set S and target t , construct the graph G , candidate coarse community C_{coa} , and the score function $\Upsilon(C)$ as described above. The task of finding a subset $S' \subseteq S$ such that the sum of its elements is t translates to finding a community $C_{opt} \subseteq G$ such that $\Upsilon(C_{opt}) = 1$. Therefore, this, in turn, is equivalent to finding an ordering $\pi \in \Pi$ such that $\Upsilon(C_{coa} + \pi) = 1$, where $C_{coa} + \pi$ represents the community obtained by applying the sequence of operations π starting from C_{coa} . (4) Equivalence of Problems: If there exists a subset $S' \subseteq S$ such that the sum of its elements is t , then there exists an ordering $\pi \in \Pi$ such that $\Upsilon(C_{coa} + \pi) = 1$. Conversely, if there exists an ordering $\pi \in \Pi$ such that $\Upsilon(C_{coa} + \pi) = 1$, then the corresponding subset of nodes in G (which represents the elements of S) has a sum of exactly t .

Since the subset sum problem (decision version) is NP-hard [47], and we have shown that it can be reduced to the community refinement problem in polynomial time, it follows that the community refinement problem is also NP-hard. ■

There are two primary motivations for employing reinforcement learning (RL for short) in our approach. First, since the community refinement problem is NP-hard (Theorem 3), rendering direct solutions impractical. Second, our goal is to achieve adaptive node selection during the refinement stage,

eliminating the need to establish a predetermined threshold for nodes or fix the resulting community size (details in Section VI-B). Fortunately, reinforcement learning naturally has the advantage of solving the above two issues [28]–[30]. Thus, we can formalize the community refiner component as a Markov Decision Process (MDP), characterized by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ (Section II-C), which is stated as follows:

- **State.** During the refinement process, we employ the trained state encoder to encode both the community and its neighbors, which subsequently serves as the input for the state representation. We define the initial state $\mathcal{S}_1$ as follows:

$$\mathcal{S}_1(u) = f_s(\mathcal{F}[u], \mathbb{I}\{u \in C_{coa}\}), \text{ where } u \in C_{coa} \cup \partial C_{coa}. \quad (14)$$

Here, $\partial C_{coa} = \{v \in V \setminus C_{coa} | \exists u \in C_{coa}, (u, v) \in E\}$ is the boundary of C_{coa} . $\mathbb{I}$ represents the indicator vector, where it takes the value of 1 when node u is present in the community C_{coa} and 0 otherwise.

- **Action.** At each step of this process, we consider the addition of a node from the current state, particularly from the neighbors of the intermediate community, while concurrently removing a node from the community. Ideally, these two operations should operate independently. To achieve this, we design two distinct prediction networks tailored to these different strategies. We utilize multilayer perceptrons (MLP) as the policy networks, which decode the state vector into scalar score values. Specifically, at step t , the two policy networks, f_ϕ and f_φ , evaluate the action spaces of the two strategies by assigning scores, respectively:

$$Score(u) = \begin{cases} f_\phi(\mathcal{S}_t(u)), & u \in \partial C_t \\ f_\varphi(\mathcal{S}_t(u)), & u \in C_t \end{cases} \quad (15)$$

∂C_t and C_t denote the respective action spaces for node addition and node removal. Separate policy networks are utilized to independently predict the action scores within each of these spaces.

- **State Transition.** Upon obtaining the scores, we identify the node with the highest score from ∂C_t to be appended into the community, while concurrently removing the node with the lowest score from C_t . This procedure yields the new community C_{t+1} . We formulate the state transition as follows:

$$\mathcal{S}_{t+1}(u) = f_s(\mathcal{S}_t(u), \mathbb{I}\{u \in C_{t+1}\}), \text{ } u \in C_{t+1} \cup \partial C_{t+1}. \quad (16)$$

- **Reward.** Our main goal is to maximize the score value of the community (Definition 1). To achieve this, we define the reward function as the variation in the score value of the target community (i.e., $Obj(\cdot)$) [36], [48]–[51]. More specifically,

$$\mathcal{R}_t = Obj(C_{t+1}) - Obj(C_t). \quad (17)$$

The reward function $\mathcal{R}_t$ described above introduces a substantial enhancement to the objective function. This formulation strategically guides the model to prioritize improvements in community value, particularly within regions exhibiting high objective function values, thereby facilitating convergence toward the global optimum. Furthermore, the proposed reward mechanism enhances the flexibility of our framework, rendering it adaptable to any objective function. For networks with available ground-truth labels, established evaluation metrics

such as ARI and F1 can be employed. Conversely, for real-world networks without labels, the framework can be guided by user-specified objectives, such as the proposed attribute-augmented conductance, thereby facilitating the fine-tuning of communities to achieve superior structural and attribute-aligned configurations.

Moreover, we propose enhancing the reward mechanism to better address the characteristics of labeled networks:

$$r_{label} = \begin{cases} 1 & \text{if the appending is in } \hat{C} \\ -1 & \text{otherwise} \end{cases} \quad (18)$$

Here, $\hat{C}$ represents the ground-truth community. Accordingly, the reward function is revised to $\mathcal{R}_t + r_{label}$.

The described above specifically applies to the node addition process. Conversely, during the node removal process, the condition for r_{label} is modified if the removal is not in $\hat{C}$. This adjustment ensures that the reward mechanism appropriately encourages the elimination of nodes that are not relevant to the community.

Flexible termination strategies. Existing approaches often rely on a community scoring threshold or limit the number of nodes [14], [15], are unsatisfactory due to their restrictive nature. Inspired by these, we introduce two flexible and smooth termination signals, s_a and s_r , to notify the two strategy networks, respectively. Taking the appending process as an example, we conceptualize the termination signal s_a as a virtual node, with its features being randomly generated, and incorporate it into the prediction along with the other node features. The appending process terminates when our RL model selects the s_a node. Analogously, the removing process terminates when the model selects the predefined termination node s_r . Upon termination of both processes, the resulting state is considered the final community. The flexible termination strategy is evaluated in Section VI-B.

Offline training. Figure 4 illustrates the details of the offline training. Specifically, during the offline training phase, we initially sample a candidate coarse community from the training dataset and permit the agent to execute the complete community refinement process. Specifically, the agent begins by encoding the state of the community (Eq. (14), illustrated by the color bar in the figure), and subsequently employs the policy network to decode this state into score values (Eq. (15), represented by the blue and green bars), which are then utilized to calculate the reward and update the state. It is crucial to emphasize that, to strike a balance between exploration and exploitation during training, we adopt an ϵ -greedy action strategy. Under this strategy, there is a probability of $1 - \epsilon$ that we will select the node with the highest (or lowest) score, while with a probability of ϵ , a random action is taken. The value of ϵ is linearly annealed from 1.0 to 0.05 throughout the refinement process. The refinement process is repeated iteratively until the predefined termination strategies are satisfied, resulting in the formation of a trajectory $\mathcal{T}$. During each episode, a single trajectory is used for iterative training, and this procedure continues over τ episodes until model convergence.

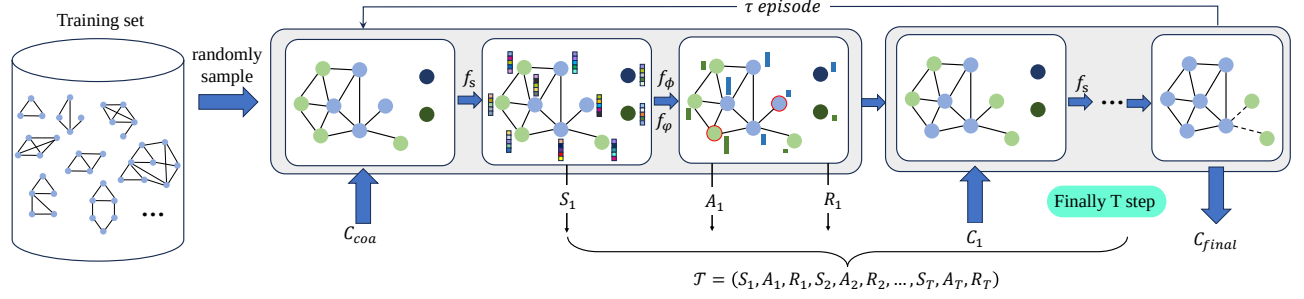


Fig. 4. Offline training. Initiating the refinement process from the candidate coarse community, each step generates a new set of states, actions, and rewards, continuing until the predefined termination policy is activated. This procedure is conducted over τ episodes, during which the agent is trained utilizing the τ trajectories obtained (best review in color).

Policy gradient method [52] is a well-known approach to strategy learning, including Trust Region Policy Optimization (TRPO) [30] and Proximal Policy Optimization (PPO) [28], [29]. In this paper, we adopt PPO as the key component of our model due to its ability to stabilize the training process by imposing constraints on the policy updates, thus preventing large, destabilizing changes. The loss function for the PPO is delineated below:

$$\mathcal{L}(\phi) = \mathbb{E}_t [\min(r_t(\phi)A_t, \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon)A_t)]. \quad (19)$$

$r_t(\phi) = \frac{f_{\phi_{new}}}{f_{\phi_{old}}}$ represents the ratio of the new policy to the old policy. A_t denotes the advantage function, which is typically estimated using temporal differencing methods. The clipping function $\text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon)$ constrains the policy ratio $r_t(\phi)$ within the interval $[1 - \epsilon, 1 + \epsilon]$. Specifically, if $r_t(\phi)$ is less than $1 - \epsilon$, it is set to $1 - \epsilon$; if $r_t(\phi)$ exceeds $1 + \epsilon$, it is set to $1 + \epsilon$. This clipping mechanism is essential to the loss function, as it prevents excessively large updates to the policy during reinforcement learning training, thereby mitigating the risk of instability in the training process.

Online refinement. The online refinement commences with the agent encoding the candidate community and its neighbors into embedding vectors through the state encoder. These vectors are then decoded into node scores using the policy network. Distinct from the training phase, this stage does not involve exploratory actions; instead, the agent directly selects nodes with the highest (or lowest) scores to construct the new community. This iterative process stops until a predetermined termination criterion is met.

Remark. We further elaborate on the advantages of our graph optimization task compared to node classification tasks, such as those represented by TransZero. When TransZero treats community search as a node classification task, it requires calculating the similarity of all nodes relative to the query node and then assessing changes in community quality, halting the search immediately when the quality declines. Note that community quality is determined by a function proposed by TransZero, which is limited by the effectiveness of the function itself. In contrast, we employ a reinforcement learning approach to learn strategies for adding or removing nodes and conditions for stopping the search from training data. The learned strategies determine which nodes to add or remove

TABLE III
STATISTICS OF DATASETS.

Dataset	$ V $	$ E $	$ F $	$ C $	$Mean(C)$	$Mean(d)$
Cora [18]	2,708	10,556	1,433	7	386.86	3.89
Facebook [53]	3,622	72,964	317	130	15.62	40.29
Cocs [18]	18,333	163,788	6,806	15	1,222.2	8.93
Twitter [53]	87,760	1,293,985	27,201	2,838	10.88	29.49
Amazon [53]	13,178	3,3767	658	4,517	9.31	5.12
Youtube [53]	216,544	1,393,206	2,165	2,865	7.67	12.86

and when to stop the search, without any explicit constraints. Compared to node classification, graph optimization tasks can adapt to different types of networks and flexibly discover high-quality communities.

VI. EXPERIMENTAL EVALUATION

In this section, we conduct comprehensive experiments to evaluate the effectiveness and efficiency of our solutions. All experiments are conducted on a Linux machine with an Intel Xeon(R) Silver 4210 @2.20GHz CPU and 1 TB RAM.

A. Experimental Setup

Datasets. Our solutions have been rigorously evaluated on six publicly available datasets with ground-truth communities (Table III), which serve as well-established benchmarks for the community search problem [2], [4], [5], [14]–[19], [42], [45], [46]. In these datasets, $|F|$ represents the number of attributes, $|C|$ denotes the number of communities, $Mean(C)$ indicates the average size of the communities, and $Mean(d)$ reflects the average degree of the nodes. We classify the ground-truth communities into three categories: training, validation, and testing communities. The training communities are utilized for model training, the validation communities for fine-tuning parameters, and the testing communities for assessing the model's performance. Consistent with previous research [14]–[19], we also randomly allocate these categories in 5:1:4.

Algorithms. The following ten baselines are compared. (1) Traditional rule-based community search methods: CTC [3] based on k -truss, CST [2] based on k -core, M k ECS [4] based on k edge connected component, and conductance [9]–[11]. Note that we do not include density [6]–[8] and modularity [5] in the experiments because they ignore the external sparsity and are outperformed by conductance [24], [25]. (2) Learning-based methods: ICS-GNN [14], QD-GNN [15], COCLEP [16]

CGNP [17], CommunityAF [19] TransZero [18]. NCSAC is our proposed algorithm. There is no comparison with *ALICE* [27], as the authors did not provide the source code, and reproducing the results is highly challenging. However, we discuss its drawbacks in Sections I and VII.

Metrics. Several well-established metrics are used to evaluate the quality of the identified communities [5], [16], [18]: the F1-score [54], Normalized Mutual Information (NMI) [55], and Jaccard similarity (JAC) [56]. Higher values for the F1-score, NMI, and JAC signify better performance. We also report the runtime to test the efficiency. Unless otherwise specified, for enhanced reliability, we select one vertex at random from each ground-truth community to serve as the query vertex and report the average runtime and quality.

Parameters. Unless specified otherwise, we take the default parameters of the ten baselines. On the other hand, our model NCSAC involves three main parameters: β (the preference parameter in Eq. 6), α (the loss function parameter in Eq. 11), and τ (the number of episodes in offline training of Section V-B). We set $\beta = 0.2$, $\alpha = 0.5$, $\tau = 2000$ by default.

B. Effectiveness Evaluation

Exp-1: Comparison of our algorithm NCSAC with baselines. Figure 5 provides a detailed box plot showing the F1-score, NMI, and JAC values achieved by different algorithms across all datasets. Besides, Table IV also shows the median scores of different methods for a clearer comparison. Note that we only explain the F1-score because NMI and JAC have similar results. For F1-score, we have the following observations: (1) Our model *NCSAC* achieves the best median scores on five of the six datasets (on Cocs, *NCSAC* is slightly worse than TransZero), with F1-score improvement of 5.3%~42.4%. For example, on Facebook, the F1-score of *NCSAC* is 0.47, whereas the runner-up is 0.33, representing an improvement of 42.4% in the F1-score. This is because our model integrates the strengths of both rule-based and learning-based methodologies. However, baseline methods study rule-based approaches and learning-based approaches independently, resulting in suboptimal community quality. (2) Our proposed *NCSAC* consistently outperforms all baselines in most cases, except for being slightly less effective than QD-GNN and COCLEP on Facebook, in terms of the quality lower bound. This outcome is attributed to *NCSAC*'s robust two-stage design. Specifically, baseline methods typically add nodes sequentially from a query node based on predefined strategies, which can be susceptible to noise, resulting in early termination and suboptimal community quality. Essentially, some nodes that could potentially improve the community score are excluded due to the limitations of these models. In contrast, *NCSAC* initially utilizes a novel attribute-augmented conductance method to identify a satisfactory and interpretable candidate coarse community. This community is then refined using reinforcement learning, ensuring a more consistent and superior performance. (3) *NCSAC* exhibits stability that is on par with (indicated by a narrower box width) other learning-based methods. Besides, learning-based models are consistently better than traditional rule-based models (e.g., CTC,

CST, and MKECS) across all datasets. This can be attributed to the fact that learning-based models possess the flexibility to capture both structural proximity and attribute similarity within communities, a feat that traditional rule-based models are unable to achieve. In short, these results give clear evidence that our model can find higher-quality communities when contrasted with baselines.

Exp-2: Comparison of the traditional conductance and the attribute-augmented conductance. Table V offers a comparative analysis between traditional conductance and the newly introduced attribute-augmented conductance. To isolate the evaluation to the community extraction process alone, we report the quality of the candidate coarse community without considering the subsequent refinement stage. As can be seen, the results demonstrate that our proposed attribute-augmented conductance consistently identifies more accurate communities compared to traditional conductance. Remarkably, attribute-augmented conductance achieves an average improvement of 0.1028 in F1-score, 0.0749 in NMI, and 0.0784 in JAC across all datasets, highlighting its exceptional ability to discern community structures.

Exp-3: Performance of NCSAC with varying parameters.

This experiment investigates how the parameters affect the performance of our *NCSAC*. Figure 6 only reports the F1-score, with analogous trends observed on NMI and JAC. Specifically, we have the following conclusions: (1) Figure 6 (a) presents the results obtained by varying β , which strikes a balance between topology-based conductance and attribute-based conductance in Equation 6. As depicted, on five out of six datasets (with Amazon being an exception showing no significant change with β), the performance of *NCSAC* diminishes as β rises. This trend can be intuitively understood as follows: as β increases, the weight assigned to topological factors grows, thereby overshadowing the influence of attribute-based factors and subsequently resulting in a decline in quality. This finding highlights the pivotal role of attribute similarity and validates the effectiveness of our proposed attribute-augmented conductance. (2) Figure 6 (b) illustrates the performance of *NCSAC* with varying α , which serves to balance the triplet loss and contrastive loss in Equation 11. As can be seen, The F1-score remains relatively stable as α increases, indicating that the impact of the triplet loss significantly exceeds that of the contrastive loss. Subsequent ablation studies provide further support for this observation. (3) Figure 6 (c) presents the influence of the number of episodes τ on the F1-score. As observed, the F1-score experiences a rapid increase (by 0.2-0.3) within the initial 800 episodes, followed by a gradual slowdown (of less than 0.05) in the subsequent 800-2000 episodes. This finding suggests that a relatively small number of episodes are sufficient to achieve a high F1-score. (4) Let η be the fixed number of nodes refined during the refinement phase (section V-B) and $RFS = \frac{F1\text{-score obtained by FixNCSAC}}{F1\text{-score obtained by NCSAC}}$ be the relative F1-score, where FixNCSAC is the *NCSAC* with the fixed number of nodes refined. This experiment studies the effectiveness of the proposed flexible termination strategy (details in Section V-B). Figure 6 (d) shows the results. As

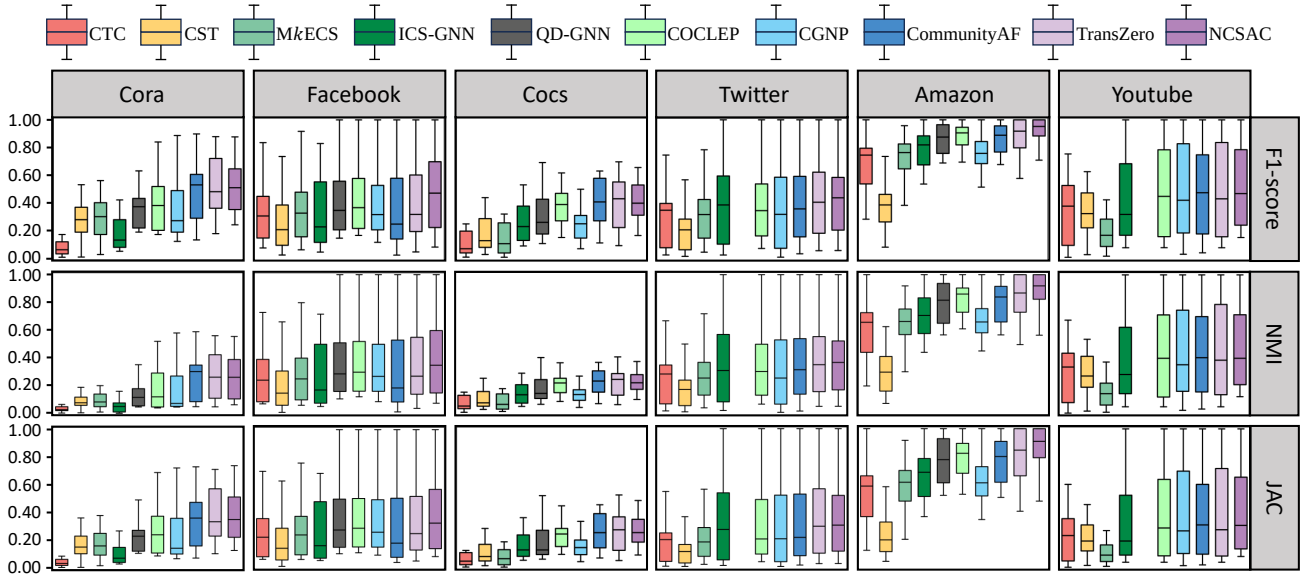


Fig. 5. Comparison of our algorithm NCSAC with baselines. Since QD-GNN runs out of memory on Twitter and Youtube, we only report their results on the four remaining datasets (best view in color).

TABLE IV
MEDIAN SCORES OF DIFFERENT METHODS. OOM INDICATES THAT THE CORRESPONDING METHODS RAN OUT OF MEMORY.

F1/NMI/JAC	Cora	Facebook	Cocs	Twitter	Amazon	Youtube
CTC	0.05/0.03/0.02	0.30/0.22/0.23	0.06/0.04/0.03	0.34/0.20/0.21	0.74/0.58/0.65	0.38/ 0.34/0.23
CST	0.27/0.14/0.07	0.21/0.14/0.14	0.14/0.07/0.09	0.20/0.11/0.12	0.38/0.20/0.29	0.31/0.26/0.20
MKECS	0.29/0.15/0.08	0.32/0.23/0.24	0.13/0.05/0.07	0.31/0.18/0.19	0.76/0.61/0.66	0.18/0.14/0.10
ICS-GNN	0.12/0.06/0.04	0.22/0.15/0.16	0.22/0.12/0.13	0.38/0.27/0.28	0.81/0.68/0.70	0.31/0.27/0.20
QD-GNN	0.36/0.22/0.11	0.34/0.27/0.28	0.25/0.13/0.13	OOM	0.87/0.77/0.81	OOM
COCLEP	0.37/0.23/0.11	0.36/0.28/0.29	0.39/0.21/0.21	0.34/0.20/0	0.90/0.82/0.86	0.44/0.40/0.30
CGNP	0.26/0.14/0.07	0.31/0.25/0.26	0.25/0.14/0.16	0.31/0.20/0.21	0.75/0.61/0.65	0.41/0.37/0.27
CommunityAF	0.52/0.35/0.28	0.25/0.17/0.18	0.40/0.22/0.25	0.37/0.31/0.23	0.88/0.80/0.83	0.44/0.40/0.31
TransZero	0.47/0.33/0.26	0.32/0.24/0.26	0.43/0.24/0.29	0.40/0.34/0.30	0.91/0.84/0.86	0.42/ 0.38/0.29
NCSAC	0.50/0.34/ 0.29	0.47/0.32/0.34	0.42/0.23/0.27	0.41/0.35/0.31	0.94/0.89/0.91	0.46/0.41/0.33

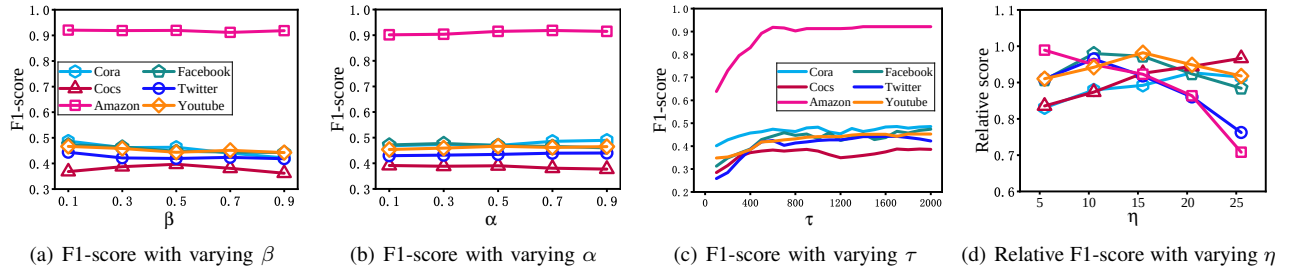


Fig. 6. Performance of NCSAC with varying parameters.

TABLE V
COMPARISON OF THE TRADITIONAL CONDUCTANCE AND THE ATTRIBUTE-AUGMENTED CONDUCTANCE.

	Traditional Conductance			Attribute-augmented Conductance		
	F1-score	NMI	JAC	F1-score	NMI	JAC
Cora	0.2195	0.0712	0.1348	0.3721	0.1281	0.2486
Facebook	0.3059	0.2403	0.2191	0.4145	0.3171	0.2938
Cocs	0.2562	0.0858	0.1522	0.3207	0.1580	0.2121
Twitter	0.2292	0.1968	0.1428	0.3647	0.3033	0.2469
Amazon	0.8666	0.8139	0.7752	0.8892	0.8381	0.8038
Youtube	0.2664	0.2380	0.1988	0.3995	0.3511	0.2882
Average +/-	-	-	-	+0.1028	+0.0749	+0.0784

illustrated, *RFS* consistently remains below 1 and exhibits no discernible pattern of variation across all datasets. Thus, this result provides compelling evidence that our proposed flexible

termination strategy consistently outperforms the fixed-node refinement strategy.

C. Efficiency Evaluation

Exp-4: Runtime of different algorithms. Figure 7 depicts the runtime of various algorithms across all datasets. The observations are as follows: (1) Traditional rule-based methods (namely CTC, CST, and MKECS) exhibit consistently faster execution speeds compared to learning-based approaches, albeit at the expense of significantly compromised quality, as discussed in Section VI-B. This is attributable to their reliance on predefined structural constraints for localized community

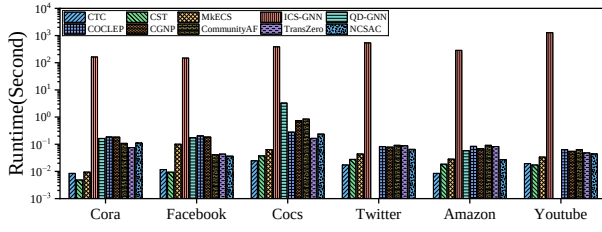


Fig. 7. Runtime of different algorithms (best view in color).

TABLE VI

EFFICIENCY OF ADAPTIVE COMMUNITY EXTRACTOR.

Dataset	Cora	Facebook	Cocs	Twitter	Amazon	Youtube
NAC	0.3128	0.1174	3.8346	24.8306	0.0231	4.1999
ACE	0.02	0.0097	0.1153	0.0131	0.0003	0.0338

search, eliminating the need for intricate learning processes. However, the inflexibility of these patterns undermines the quality of the results. (2) Our proposed model, *NCSAC*, demonstrates runtime efficiency comparable to other learning-based methods. Notably, *NCSAC* substantially surpasses alternative approaches on datasets with well-defined structures, such as Amazon and Twitter. This superiority stems from the efficacy of our community extractor, which swiftly identifies candidate community structures, thereby minimizing the necessity for extensive refinement. Conversely, in datasets with less discernible community structures, like Cora and Cocs, TransZero shows faster query times due to its simpler termination condition, which ceases the search upon the initial decline in community score. In contrast, CommunityAF employs a more elaborate termination criterion based on a sliding window approach, leading to longer query time. (3) ICS-GNN emerges as the most time-intensive method, as it necessitates retraining for each individual query, equating its query time to the training time. Consequently, ICS-GNN incurs the highest computational cost among all evaluated methods. In summary, these results give some preliminary evidence that our model is indeed efficient in practice.

Exp-5: Efficiency of adaptive community extractor. To evaluate the efficiency of our adaptive community extractor (i.e., *ACE* in Algorithm 1), we implement the naive method, named *NAC* (discussed as a failed attempt in Section IV). Namely, *NAC* materializes the dense multigraph (Figure 2) for computing the attribute-augmented conductance. As shown in Table VI, our proposed *ACE* is at least $12\times$ faster than the naive method *NAC*. Specifically, *ACE* is $15\times$, $12\times$, $33\times$, $1895\times$, $77\times$, $124\times$ faster than *NAC* on Cora, Facebook, Cocs, Twitter, Amazon, and Youtube respectively. This is because our *ACE* requires only linear time complexity (Theorem 2), whereas the worst-case scenario for *NAC* entails a quadratic time complexity. Thus, these results align with our theoretical analysis (Section IV).

Exp-6: Scalability testing. To further evaluate the scalability of our adaptive community extractor, we first generate 60 synthetic subgraphs by randomly sampling 20%, 40%, 60%, 80%, and 100% of vertices or edges from each of the six datasets. Subsequently, we present the runtime on these subgraphs in Figure 8. As depicted, the runtime of Algorithm 1 scales nearly linearly with the size of the subgraphs. These findings suggest

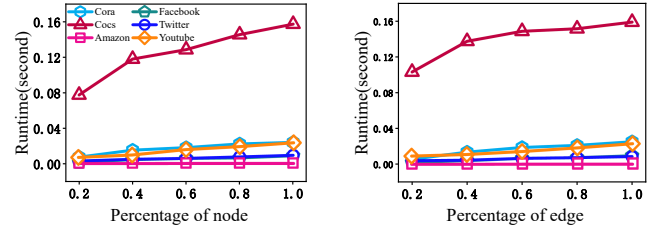


Fig. 8. Scalability testing.

that our proposed algorithm can deal with massive graphs.

D. Ablation Studies and Case Studies

Exp-7: Ablation studies. Figure 9 illustrates the ablation studies of four key components of our *NCSAC*. Specifically, *NCSAC w/o Refiner* is the *NCSAC* without the community refinement stage, *NCSAC w/o r_{label}* denotes training the two policy networks without using the reward enhancement mechanism r_{label} , *NCSAC w/o L_C* involves training the state encoder without incorporating the contrastive loss L_C , *NCSAC w/o L_T* trains the state encoder without the triplet loss L_T , *NCSAC-PG* substitutes the PPO algorithm with a traditional policy gradient (PG) method. Note that we only explain the F1-score because NMI and JAC have similar results. For the F1-score, we have the following conclusions: (1) *NCSAC vs. NCSAC w/o Refiner*. The community refinement module significantly improves the quality of the candidate coarse community, fulfilling its intended purpose. Specifically, the refinement module improved by at least 0.05 across all datasets, with the most significant improvement reaching up to 0.1 on the Cora. (2) *NCSAC vs. NCSAC w/o r_{label}* . The reward enhancement mechanism r_{label} is designed to facilitate the convergence of the policy networks, leading to consistent improvements in community quality across all six datasets. Notably, it achieves a gain of 0.08 on Cocs and 0.01 on Amazon, which aligns well with our design intuition. The effectiveness of this mechanism becomes particularly evident in large communities, where the $Obj(\cdot)$ (Eq. (17)) change caused by adding a single node is marginal, making convergence difficult. In such cases, r_{label} provides additional guidance that stabilizes the training process. (3) *NCSAC vs. NCSAC w/o L_C & NCSAC w/o L_T* . Within the community-aware state encoder (Section IV-B), the contrastive loss and triplet loss can indeed effectively incorporate community awareness into the embedded space of nodes (Eq. 11). In particular, triplet loss L_T plays a more substantial role in enhancing state representations compared to contrastive loss L_C . For example, on Cora, the F1-score of *NCSAC w/o L_T* is 0.41, while that of *NCSAC w/o L_C* is 0.49. (4) *NCSAC vs. NCSAC-PG*. In the reinforcement learning component, the proximal policy optimization (as outlined in Eq. 19) demonstrates an average improvement of 0.2 compared to PG across five of the six datasets. Nonetheless, on the Twitter dataset, PG outperforms PPO by approximately 0.1, which we postulate might be attributed to overfitting in PPO under the same sampling conditions. (5) The full set of components (i.e., *NCSAC*) yields the best performance, indicating the effectiveness of these

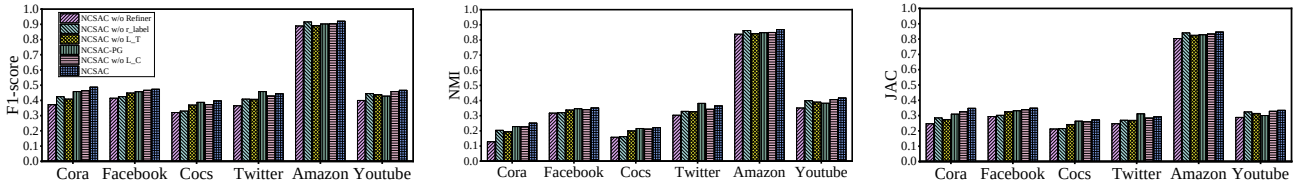


Fig. 9. Ablation studies (best view in color).

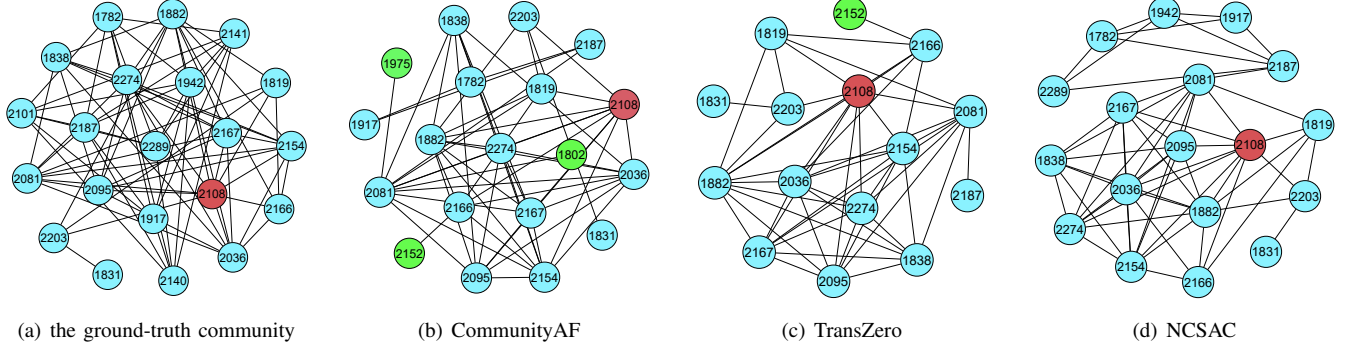


Fig. 10. Case studies. The red node represents the query node, while the blue node belongs to the ground-truth community and the green node does not.

components in enhancing algorithm performance. In summary, these results provide some preliminary evidence that the four key components we have designed are practical and effective in real-world applications.

Exp-8: Case studies. We conduct case studies on Facebook [53] with the ground-truth community to further evaluate the utility of the proposed NCSAC. Specifically, we choose node 2108 as the query node (marked in red), and the communities identified by different models are visualized in Figure 10. As can be seen, the communities identified by both CommunityAF and TransZero include nodes (depicted in green) that do not belong to the ground-truth community. Besides, they also miss many nodes within the ground-truth community. Consequently, their F1-scores are 0.8 and 0.77, respectively. In contrast, our NCSCA exclusively identifies nodes from the ground-truth community, with only three nodes from the ground-truth community being undetected. Thus, NCSCA achieves the highest congruence with the ground-truth community, boasting an impressive F1-score of 0.92. This result can be explained as follows: TransZero employs a stricter termination condition, halting the search once the community quality starts to diminish. While this prevents over-expansion, it also restricts the exploration of potentially relevant nodes. CommunityAF, on the other hand, utilizes a sliding window mechanism, enabling it to explore a broader range of promising nodes but at the expense of including more irrelevant ones. However, our NCSCA adopts the more flexible termination strategy (details in Section VI-B), which allows it to align closely with the ground-truth community.

VII. RELATED WORK

Rule-based Community Search. Community search has emerged as an essential instrument for revealing the local structures within networks [1]. Numerous models have been proposed in the literature, which can be categorized into two main types: subgraph-based and optimization-based. Specifically, k -core [2], [13], [40], [57]–[60], k -truss [3], [41], [61], and k -ecc [4] are the representative subgraph-based models,

which restrict the local connectivity of nodes or edges. For example, k -core is a subgraph such that each node has at least k -neighbors within the subgraph. On the other hand, density [6]–[8], modularity [5], and conductance [9]–[11], [25] are the representative optimization-based models, which aim to achieve a globally optimal objective function. However, these rule-based community search models suffer from the following defects: (1) They (excluding conductance) focus on internal cohesion and neglects external sparsity, which violates the natural intentions of the community [24]–[26]. (2) Conductance is defined based on structure, thus neglecting important node attributes. (3) They are plagued by structural inflexibility, wherein the ground-truth community may not align with user-predefined constraints, thereby resulting in suboptimal community quality [14]–[19].

Learning-based Community Search. With the swift progress of artificial intelligence, researchers have commenced the integration of Graph Neural Network (GNN) into community search. For instance, *ICS-GNN* [14] pioneered the GNN-based approach to community search by iteratively identifying the target community with maximum GNN scores after crawling online subgraphs. *QD-GNN* [15] combined local query dependency structures with global graph embedding. *CommunityAF* [19] utilized autoregressive flow to generate communities and can learn various community patterns. *CGNP* [17] and *CO-CLEP* [16] leveraged meta-learning and contrastive learning to solve the community search problem, respectively. However, all of these approaches completely overlook the traditional rule-based constraints. Recently, *ALICE* [27] and *TransZero* [18] effectively integrate traditional rule-based subgraph metrics with GNN, resulting in substantial improvements. At a high level, they first extracted a predefined new subgraph G_{new} , then identified the target community by executing the node classification task within G_{new} . Unfortunately, they are heavily dependent on the performance of G_{new} , since nodes outside G_{new} have no chance to contribute to the target community, resulting in poor quality (Section 6). However, our

method focuses on optimizing communities by strategically removing and adding nodes, starting from G_{new} .

VIII. CONCLUSION

In this paper, we propose a novel Neural Community Search via Attribute-augmented Conductance, termed *NCSAC*, to integrate deep learning techniques with traditional rule-based constraints, thereby improving the quality of community search. Specifically, *NCSAC* utilizes the proposed attribute-augmented conductance to adaptively extract a coarse candidate community with satisfactory quality. Subsequently, *NCSAC* refines the candidate community through advanced reinforcement learning techniques, ultimately producing high-quality results. Extensive experiments are conducted on six real-life graphs, in comparison with ten competitors, to demonstrate the superiority of our proposed solution. The following two aspects of future work are worth exploring. (1) Given that real-world graphs frequently exhibit characteristics such as dynamism, heterogeneity, heterophily, or even noise, exploring ways to extend our algorithm to accommodate these more intricate scenarios represents a promising research direction. (2) We aim to devise multiple parallel strategies to bolster the scalability of our algorithms.

REFERENCES

- [1] Y. Fang, X. Huang, L. Qin, Y. Zhang, W. Zhang, R. Cheng, and X. Lin, "A survey of community search over big graphs," *The VLDB Journal*, vol. 29, pp. 353–392, 2020.
- [2] W. Cui, Y. Xiao, H. Wang, and W. Wang, "Local search of communities in large graphs," in *SIGMOD*, 2014, pp. 991–1002.
- [3] X. Huang, L. V. S. Lakshmanan, J. X. Yu, and H. Cheng, "Approximate closest community search in networks," *Proc. VLDB Endow.*, vol. 9, no. 4, pp. 276–287, 2015.
- [4] T. Akiba, Y. Iwata, and Y. Yoshida, "Linear-time enumeration of maximal k-edge-connected subgraphs in large networks by random contraction," in *CIKM*, 2013, pp. 909–918.
- [5] J. Kim, S. Luo, G. Cong, and W. Yu, "Dmcs: Density modularity based community search," in *SIGMOD*, 2022, pp. 889–903.
- [6] Y. Wu, R. Jin, J. Li, and X. Zhang, "Robust local community detection: On free rider effect and its elimination," *Proc. VLDB Endow.*, vol. 8, no. 7, pp. 798–809, 2015.
- [7] Y. Dai, M. Qiao, and L. Chang, "Anchored densest subgraph," in *SIGMOD*, 2022, pp. 1200–1213.
- [8] X. Ye, R. Li, L. Liang, Z. Liu, L. Lin, and G. Wang, "Efficient and effective anchored densest subgraph search: A convex-programming based approach," in *KDD*, 2024, pp. 3907–3918.
- [9] R. Andersen, F. R. K. Chung, and K. J. Lang, "Local graph partitioning using pagerank vectors," in *FOCS*, 2006, pp. 475–486.
- [10] K. Kloster and D. F. Gleich, "Heat kernel based community detection," in *KDD*, 2014, pp. 1386–1395.
- [11] L. Lin, R. Li, and T. Jia, "Scalable and effective conductance-based graph clustering," in *AAAI*, 2023, pp. 4471–4478.
- [12] X. Xie, S. Liu, J. Zhang, S. Han, W. Wang, and W. Yang, "Efficient community search based on relaxed k-truss index," in *SIGIR*, 2024, pp. 1691–1700.
- [13] Y. Li, G. Zang, C. Song, X. Yuan, and T. Ge, "Leveraging semantic information for enhanced community search in heterogeneous graphs," *Data Sci. Eng.*, vol. 9, no. 2, pp. 220–237, 2024.
- [14] J. Chen, J. Gao, and B. Cui, "lcs-gnn+: lightweight interactive community search via graph neural network," *The VLDB Journal*, vol. 32, no. 2, pp. 447–467, 2023.
- [15] Y. Jiang, Y. Rong, H. Cheng, X. Huang, K. Zhao, and J. Huang, "Query driven-graph neural networks for community search: From non-attributed, attributed, to interactive attributed," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1243–1255, 2022.
- [16] L. Li, S. Luo, Y. Zhao, C. Shan, Z. Wang, and L. Qin, "Coclep: Contrastive learning-based semi-supervised community search," in *ICDE*, 2023, pp. 2483–2495.
- [17] S. Fang, K. Zhao, G. Li, and J. X. Yu, "Community search: a meta-learning approach," in *ICDE*, 2023, pp. 2358–2371.
- [18] J. Wang, K. Wang, X. Lin, W. Zhang, and Y. Zhang, "Efficient unsupervised community search with pre-trained graph transformer," *Proc. VLDB Endow.*, vol. 17, no. 9, pp. 2227–2240, 2024.
- [19] J. Chen, Y. Xia, and J. Gao, "Communityaf: An example-based community search method via autoregressive flow," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2565–2577, 2023.
- [20] Y. Wang, X. Gou, X. Xu, Y. Geng, X. Ke, T. Wu, Z. Yu, R. Chen, and X. Wu, "Scalable community search over large-scale graphs based on graph transformer," in *SIGIR*, 2024, pp. 1680–1690.
- [21] W. Cai, F. Zhu, Z. Liu, and M. Wu, "Heterocs: A heterogeneous community search system with semantic explanation," in *SIGIR*, 2023, pp. 3155–3159.
- [22] Y. Zhang, R. Li, L. Lin, Q. Zhang, L. Qin, and G. Wang, "Integral densest subgraph search on directed graphs," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 176:1–176:26, 2025.
- [23] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016.
- [24] R. Kamal and A. Bagchi, "A lovász-simonovits theorem for hypergraphs with application to local clustering," *Proc. ACM Manag. Data*, vol. 2, no. 4, pp. 190:1–190:27, 2024.
- [25] Y. He, L. Lin, P. Yuan, R. Li, T. Jia, and Z. Wang, "CCSS: towards conductance-based community search with size constraints," *Expert Syst. Appl.*, vol. 250, p. 123915, 2024.
- [26] J. Leskovec, K. J. Lang, and M. W. Mahoney, "Empirical comparison of algorithms for network community detection," in *WWW*, 2010, pp. 631–640.
- [27] J. Wang, K. Wang, X. Lin, W. Zhang, and Y. Zhang, "Neural attributed community search at billion scale," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, pp. 1–25, 2024.
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [29] N. Heess, D. Tb, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. Eslami *et al.*, "Emergence of locomotion behaviours in rich environments," *arXiv preprint arXiv:1707.02286*, 2017.
- [30] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *ICML*, 2015, pp. 1889–1897.
- [31] R. Yang, X. Xiao, Z. Wei, S. S. Bhowmick, J. Zhao, and R. Li, "Efficient estimation of heat kernel pagerank for local clustering," in *SIGMOD*, 2019, pp. 1339–1356.
- [32] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NeurIPS*, 2017, pp. 1024–1034.
- [33] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio *et al.*, "Graph attention networks," vol. 1050, no. 20, 2017, pp. 10–48 550.
- [34] Y. Yu, L. Lin, Q. Liu, Z. Wang, X. Ou, and T. Jia, "GSD-GNN: generalizable and scalable algorithms for decoupled graph neural networks," in *ICMR*, 2024, pp. 64–72.
- [35] Y. Lei, H. Pei, H. Yan, and W. Li, "Reinforcement learning based recommendation with graph convolutional q-network," in *SIGIR*, 2020, pp. 1757–1760.
- [36] Y. Deng, Y. Li, F. Sun, B. Ding, and W. Lam, "Unified conversational recommendation policy learning via graph-based reinforcement learning," in *SIGIR*, 2021, pp. 1431–1441.
- [37] S. Zheng, H. Yin, T. Chen, Q. V. H. Nguyen, W. Chen, and L. Zhao, "Dream: Adaptive reinforcement learning based on attention mechanism for temporal knowledge graph reasoning," in *SIGIR*, 2023, pp. 1578–1588.
- [38] L. Lin, T. Jia, Z. Wang, J. Zhao, and R. Li, "PSMC: provable and scalable algorithms for motif conductance based graph clustering," in *KDD*, 2024, pp. 1793–1803.
- [39] L. Lin, Z. Wang, R. Li, Q. Liu, H. Qin, and J. Zhao, "Provable higher-order graph clustering: The power of peeling-based approaches," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 9, pp. 5568–5583, 2025.
- [40] N. Barbieri, F. Bonchi, E. Galimberti, and F. Gullo, "Efficient and effective community search," *Data mining and knowledge discovery*, vol. 29, pp. 1406–1433, 2015.
- [41] E. Akbas and P. Zhao, "Truss-based community search: a truss-equivalence based indexing approach," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1298–1309, 2017.
- [42] W. Cui, Y. Xiao, H. Wang, Y. Lu, and W. Wang, "Online search of overlapping communities," in *SIGMOD*, 2013, pp. 277–288.
- [43] A. Clauset, "Finding local community structure in networks," *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, vol. 72, no. 2, p. 026132, 2005.

- [44] L. Lin, Y. He, W. Chen, P. Yuan, R.-H. Li, and T. Jia, “Effective and Efficient Conductance-based Community Search at Billion Scale,” *IEEE Transactions on Big Data*, no. 01, pp. 1–14. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TBDDATA.2025.3588028>
- [45] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, “Querying k-truss community in large and dynamic graphs,” in *SIGMOD*, 2014, pp. 1311–1322.
- [46] K. Yao and L. Chang, “Efficient size-bounded community search over large networks,” *Proceedings of the VLDB Endowment*, vol. 14, no. 8, pp. 1441–1453, 2021.
- [47] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Second Edition*. The MIT Press and McGraw-Hill Book Company.
- [48] R. Wickman, “Sparrl: Graph sparsification via deep reinforcement learning,” in *SIGMOD*, 2022, p. 2521–2523.
- [49] R. Shah, K. Jain, S. Manchanda, S. Medya, and S. Ranu, “Neurocut: A neural approach for robust graph partitioning,” in *KDD*, 2024, pp. 2584–2595.
- [50] T. Peng, W. Wu, H. Yuan, Z. Bao, Z. Pengru, X. Yu, X. Lin, Y. Liang, and Y. Pu, “Graphrare: Reinforcement learning enhanced graph neural network with relative entropy,” in *ICDE*, 2024, pp. 2489–2502.
- [51] X. Wu, Y. Xiong, Y. Zhang, Y. Jiao, C. Shan, Y. Sun, Y. Zhu, and P. S. Yu, “Clare: A semi-supervised community detection algorithm,” in *KDD*, 2022, p. 2059–2069.
- [52] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” *NeurIPS*, vol. 12, 1999.
- [53] J. Yang and J. Leskovec, “Defining and evaluating network communities based on ground-truth,” in *Proceedings of the ACM SIGKDD workshop on mining data semantics*, 2012, pp. 1–8.
- [54] Y. Sasaki *et al.*, “The truth of the f-measure. teach tutor mater, 1 (5), 1–5,” 2007.
- [55] L. Danon, A. Diaz-Guilera, J. Duch, and A. Arenas, “Comparing community structure identification,” *Journal of statistical mechanics: Theory and experiment*, vol. 2005, no. 09, p. P09008, 2005.
- [56] Y. Zhang, Y. Xiong, Y. Ye, T. Liu, W. Wang, Y. Zhu, and P. S. Yu, “Seal: Learning heuristics for community detection with generative adversarial networks,” in *KDD*, 2020, pp. 1103–1113.
- [57] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu, “Effective and efficient community search over large directed graphs,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 11, pp. 2093–2107, 2018.
- [58] M. Sozio and A. Gionis, “The community-search problem and how to plan a successful cocktail party,” in *KDD*, 2010, pp. 939–948.
- [59] L. Lin, P. Yuan, R. Li, C. Zhu, H. Qin, H. Jin, and T. Jia, “QTCS: efficient query-centered temporal community search,” *Proc. VLDB Endow.*, vol. 17, no. 6, pp. 1187–1199, 2024.
- [60] Y. Zhang, L. Lin, P. Yuan, and H. Jin, “Significant engagement community search on temporal networks,” in *DASFAA*, 2022, pp. 250–258.
- [61] H. Yang, C. Zhu, L. Lin, and P. Yuan, “Towards truss-based temporal community search,” *CoRR*, vol. abs/2410.15046, 2024.



Longlong Lin received the PhD degree from Huazhong University of Science and Technology (HUST), in 2022. He is currently an associate professor in the College of Computer and Information Science, Southwest University, Chongqing, China. His current research interests include (temporal) graph clustering and graph-based machine learning.



Quanao Li received his B.E. degree in Computer Science from Chengdu University of Technology, China, in 2022. He is currently pursuing an M.S. degree at the College of Computer and Information Science, Southwest University, China. His research interests include community search, graph clustering, and related topics in data mining.



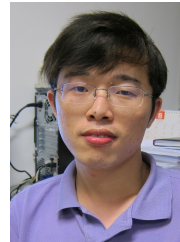
on vector data management, graph search and analytics, and brain network analytics.



Zeli Wang received the PhD degree from Huazhong University of Science and Technology (HUST), in 2022. She is now a lecturer at the College of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, China. Her current research interests include blockchain smart contract security and natural language processing security.



Jin Zhao received the PhD degree from the Huazhong University of Science and Technology (HUST), in 2022. He is currently an associate professor with the Huazhong University of Science and Technology, China. His current research interests include graph processing, system software and architecture.



Rong-Hua Li received the PhD degree from the Chinese University of Hong Kong, in 2013. He is currently a professor with the Beijing Institute of Technology (BIT), Beijing, China. His research interests include graph data management and mining, social network analysis, graph computation systems, and graph-based machine learning.



Artificial Intelligence and Data Science.

Xin Luo (Fellow, IEEE) received the B.S. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2005, and the Ph.D. degree in computer science from the Beihang University, Beijing, China, in 2011. He is currently a Professor of Data Science and Computational Intelligence with the College of Computer and Information Science, Southwest University, Chongqing, China. He has authored or coauthored over 400 papers (including over 160 IEEE Transactions/Journal papers) in the areas of



Tao Jia received the BSc degree from Nanjing University, China. He received his MSc and PhD degree from Virginia Tech, USA. He is currently a Professor at Southwest University, China. His research interest includes graph mining, brain networks, and social computing.